

Chapter-3

Agile Development

Extreme Programming XP

- Developed by Kent Beck :
 - XP is “a light-weight methodology for small to medium-sized teams developing software in the face of vague or rapidly changing requirements.”
- Alternative to “heavy-weight” software development models (which tend to avoid change and customers)
 - **"Extreme Programming turns the conventional software process sideways. Rather than planning, analyzing, and designing for the far-flung future, XP programmers do all of these activities a little at a time throughout development."**

—
-- IEEE Computer , October 1999

Embrace Change

- In traditional software life cycle models, the cost of changing a program rises exponentially over time
- A key assumption of XP is that the cost of changing a program can be hold mostly constant over time
- Hence XP is a lightweight (agile) process:
 - Instead of lots of documentation nailing down what customer wants up front, XP emphasizes plenty of feedback
 - Embrace change: iterate often, design and redesign, code and test frequently, keep the customer involved
 - Deliver software to the customer in short (2 week) iterations
 - Eliminate defects early, thus reducing costs

Four Core Values of XP

- Communication
- Simplicity
- Feedback
- Courage

Four Core Values of XP

Communication

XP emphasizes value of communication in many of its practices:

- On-site customer, user stories, pair programming, collective ownership (popular with open source developers), daily standup meetings, etc.

XP employs a *coach* whose job is noticing when people aren't communicating and reintroduce them

Four Core Values of XP

Simplicity

*"Do the simplest thing that could possibly work" (DTSTTCPW)
principle*

XP restricts developers to design only for immediate needs, rather than consider future needs. The intent is to create a simple design that can be easily implemented in code. If the design must be improved, it can be refactored at a later time.

Four Core Values of XP

Feedback

- Feedback at different time scales
- Unit tests tell programmers status of the system
- When customers write new *user stories*, programmers estimate time required to deliver changes
- Programmers produce new releases every 2-3 weeks for customers to review

Four Core Values of XP

Courage

- The courage to communicate and accept feedback
- The courage to throw code away (prototypes)
- The courage to refactor the architecture of a system

Why is it called “Extreme?”

Extreme Programming takes the effective principles and practices to extreme levels.

- Code reviews are effective as the code is reviewed all the time.
- Testing is effective as there is continuous regression and testing.
- Design is effective as everybody needs to do refactoring daily.
- Integration testing is important as integrate and test several times a day.
- Short iterations are effective as the planning game for release planning and iteration planning.

Effective Practices

Pushed to the Extreme

Extreme Programming Practices

Code Reviews

Pushed to the Extreme

Pair Programming

Testing

Pushed to the Extreme

Unit Testing &
Continuous Regression

Design

Pushed to the Extreme

Persistent Refactoring

Simplicity

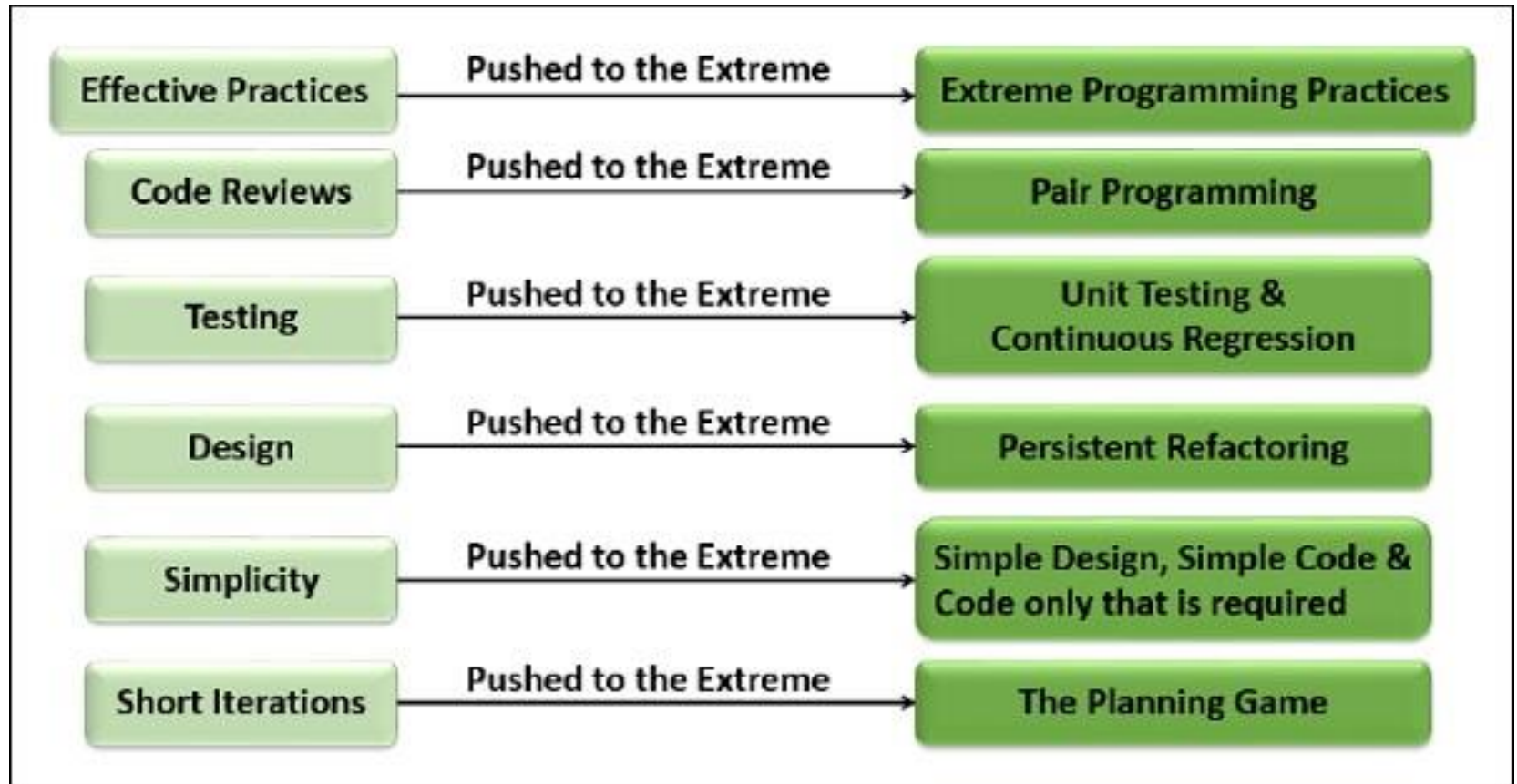
Pushed to the Extreme

Simple Design, Simple Code &
Code only that is required

Short Iterations

Pushed to the Extreme

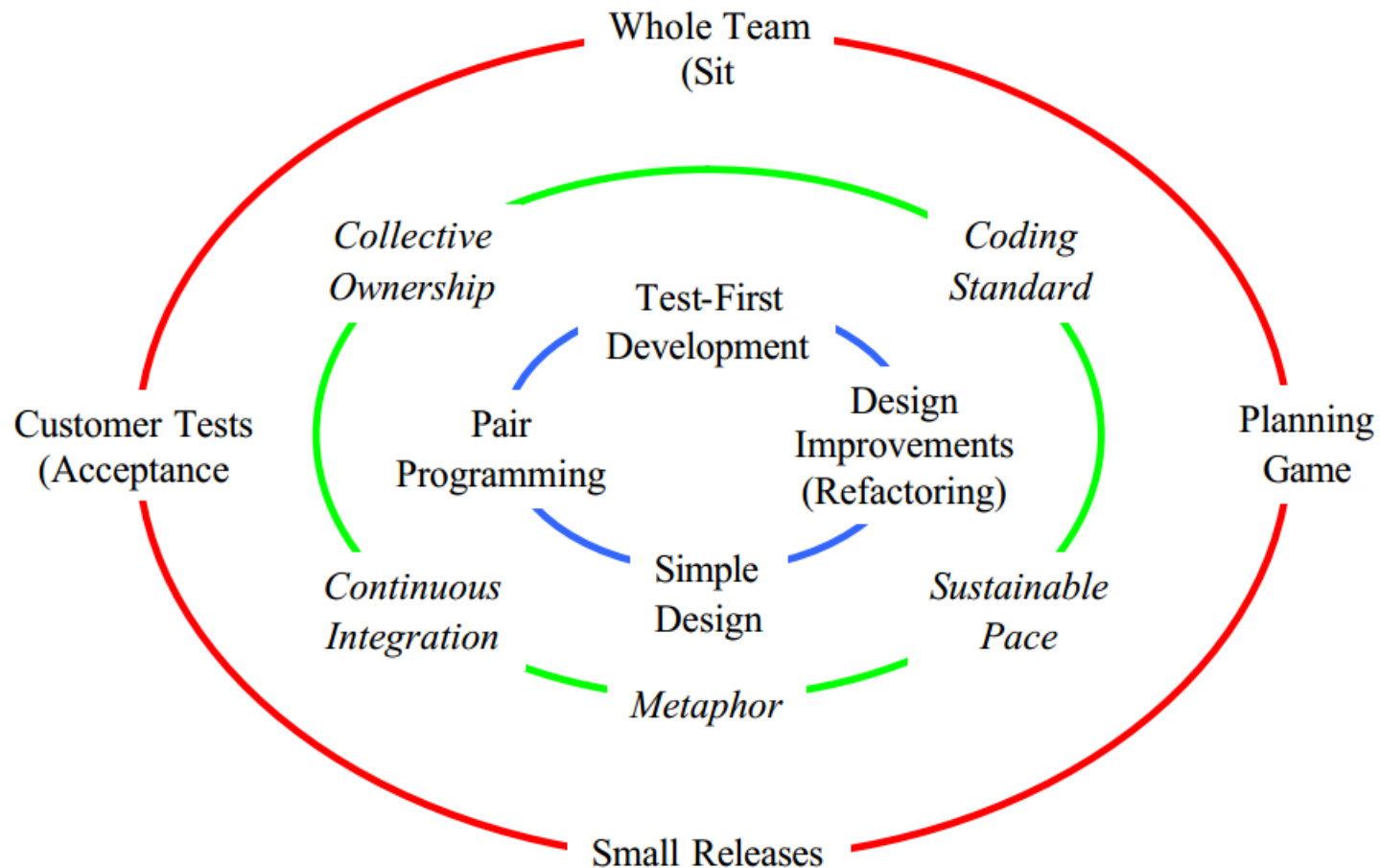
The Planning Game



Twelve XP Practices

1. *Whole Team*
2. *The Planning Game*
3. *Small Releases*
4. *Simple Design*
5. *Test-driven development*
6. *Refactoring*
7. *Pair Programming*
8. *Collective Ownership*
9. *Continuous Integration*
10. *40-Hours a Week*
11. *On-Site Customer*
12. *Coding Standards*

Twelve XP Practices



As XP is defined by values and principles, its practices also represent them and can be clustered into four groups

Group	Practices
Feedback	✓ Test-Driven Development ✓ The Planning Game ✓ On-site Customer ✓ Pair Programming
Continual Process	✓ Continuous Integration ✓ Code Refactoring ✓ Small Releases
Code understanding	✓ Simple Design ✓ Collective Code Ownership ✓ System Metaphor ✓ Coding Standards
Work conditions	✓ 40-Hour Week

Whole Team

- All the contributors to an XP project sit together, as members of one team. This team must include a business representative -- the "Customer" -- who provides the requirements, sets the priorities.
- The team will typically include testers, who help the Customer define the customer acceptance tests.
- Analysts may serve as helpers to the Customer, helping to define the requirements.
- Manager, providing resources, handling external communication, and coordinating activities.

None of these roles is necessarily the exclusive property of just one individual

Planning Game

- XP planning addresses two key questions in software development:
*predicting what will be accomplished by the due date, and
determining what to do next.*
- There are two key planning steps in XP, addressing these two questions:
 - Release Planning
 - Iteration Planning

Planning Game

- **Release Planning** is a practice where the Customer presents the desired features to the programmers, and the programmers estimate their difficulty
- With the costs estimates in hand, and with knowledge of the importance of the features, the Customer lays out a plan for the project.
- Initial release plans are necessarily imprecise:
 - neither the priorities nor the estimates are truly solid, and until the team begins to work, we won't know just how fast they will go

Planning Game

- **Iteration Planning** is the practice whereby the team is given direction every couple of weeks.
- XP teams build software in two week "iterations", delivering running useful software at the end of each iteration.
- The programmers break them down into tasks, and estimate their cost (at a finer level of detail than in Release Planning).

Small Releases

- Small releases
 - Start with the smallest useful feature set
 - Release early and often, adding a few features each time
- Simple design
 - Always use the simplest possible design that gets the job done
 - The requirements will change tomorrow, so only do what's needed to meet today's requirements

Customer Test

- As part of presenting each desired feature, the XP Customer defines one or more automated acceptance tests to show that the feature is working
- The team builds these tests and uses them to prove to themselves, and to the customers, that the feature is implemented correctly

Pair Programming

- Two programmers work together at one machine
- **Driver** enters code, while **navigator** critiques it
- Periodically switch roles

Research results:

Pair programming increases productivity

Higher quality code (15% fewer defects) in about half the time (58%)

- *Pairs produce higher quality code*
- *Pairs complete their tasks faster*
- *Pairs enjoy their work more*
- *Pairs feel more confident in their work*



Test-driven development

- **Test first:** before adding a feature, write a test for it!
- When the complete test suite passes 100%, the feature is accepted

Unit Tests automate testing of functionality as developers write it

- Each unit test typically tests only a single class, or a small cluster of classes
- Experiments show that test-driven development reduces debugging time
- Increases confidence that new features work, and work with everything

Test-driven development

- Extreme Programming is possessed with feedback, and in software development, good feedback requires good testing.
- XP teams practice "test - driven development", working in very short cycles of adding a test, then making it work
- These "programmer tests", or "unit tests" are all collected together, and every time any programmer releases any code to the repository every single one of the programmer tests must run correctly

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them.
- This improves the understandability of the software and so reduces the need for documentation.
- Changes are easier to make because the code is well-structured and clear.

Examples

- *Re-organization of a class hierarchy to remove duplicate code.*
- *Tidying up and renaming attributes and methods to make them easier to understand.*
- *The replacement of inline code with calls to methods that have been included in a program library.*

More XP Practices

- Collective code ownership
 - No single person "owns" a module
 - Any developer can work on any part of the code base at any time
 - All code gets the benefit of many people's attention, which increases code quality and reduces defects

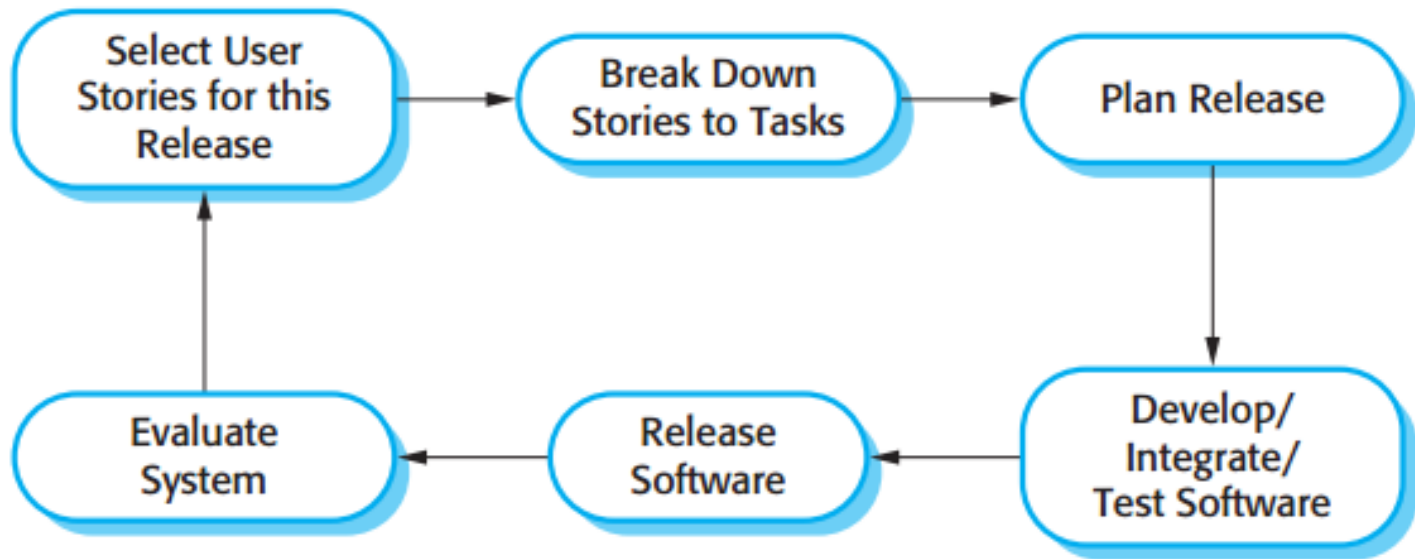
- Continuous integration
 - All changes are integrated into the code base at least daily
 - Tests have to run 100% both before and after integration

More XP Practices

- On-site customer
 - Development team has continuous access to a real live customer, that is, someone who will actually be using the system

- Coding standards
 - Everyone codes to the same standards
 - Ideally, you shouldn't be able to tell by looking at it who on the team has touched a specific piece of code

Extreme Programming Release Cycle



What is a user story?

- **User stories** are short, simple descriptions of a feature told from the perspective of the person who desires the new capability, usually a user or customer.
- In software development and product management, a user story is an informal, natural language description of one or more features of a software system. A user story is a tool used in Agile software development to capture a description of a software feature from an end-user perspective. A user story describes the type of user, what they want and why. A user story helps to create a simplified description of a requirement of the system.

User Story Examples:

- As a **[customer]**, I want **[shopping cart feature]** so that **[I can easily purchase items online]**.
- As an **[manager]**, I want to **[generate a report]** so that **[I can understand which departments need more resources]**.
- As a **[customer]**, I want to **[receive an SMS when the item is arrived]** so that **[I can go pick it up right away]**

XP Process-Planning

➤ XP Planning

- Begins with the listening, leads to creation of “**user stories**” that describes required output, features, and functionality. Customer assigns a value(i.e., a priority) to each story.
- Agile team assesses each story and assigns a **cost** (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
- Working together, stories are grouped for a **deliverable increment next release**.

XP Process-Planning

- A **commitment** (stories to be included, delivery date and other project matters) is made. Three ways:
 - *Either all stories will be implemented in a few weeks,*
 - *high priority stories first, or*
 - *the riskiest stories will be implemented first.*
- After the first increment “**project velocity**”, namely number of stories implemented during the first release is used to help define subsequent delivery dates for other increments. Customers can add stories, delete existing stories, change values of an existing story, split stories as development work proceeds.

Prescribing Medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose. If she wants to change the dose, she enters the dose and then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose and then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose and then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Examples of task cards for prescribing medication

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

XP Process-Coding

➤ XP Coding

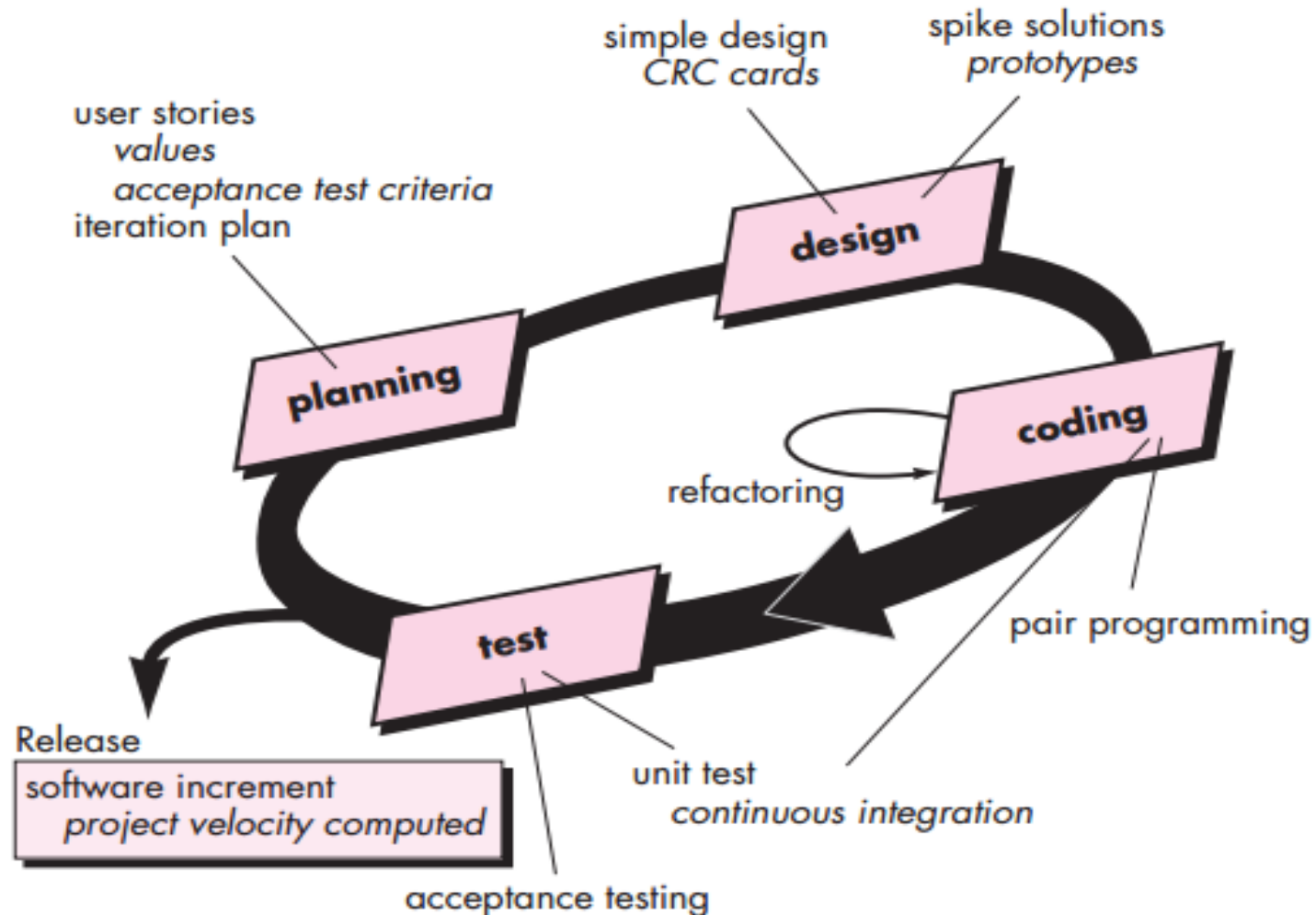
- Recommends the **construction of a unit test** for a story *before* coding commences. So implementer can focus on what must be implemented to pass the test.
- Encourages “**pair programming**”. Two people work together at one workstation. Real time problem solving, real time review for quality assurance. Take slightly different roles.

XP Process-Testing

➤ XP Testing

- All unit tests are executed daily and ideally should be automated. Regression tests are conducted to test current and previous components.
- “Acceptance tests” are defined by the customer and executed to assess customer visible functionality

XP Process



When to use XP

- **Highly-adaptive development.** XP was designed to help development teams adapt to fast-changing requirements.
- **Risky projects.** Teams applying XP practices are more likely to avoid problems connected with working on a new system, especially when a product owner sets strict deadlines for a project.
- **Small teams.** XP practices are efficient for teams that don't exceed 12 people.
- **Automated testing.** Another factor that can influence the choice of XP is the developers' ability to create and run unit tests.